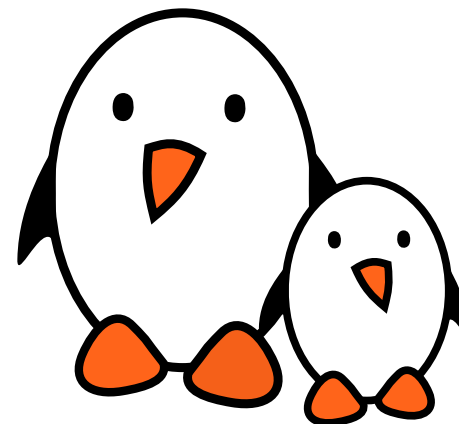




Autotools usage

bootlin





Why do we need *autotools*?

- ▶ **Portability** accross UNIX systems, architectures, Linux distributions
 - Some C functions do not exist everywhere, or have different names or prototypes, can behave differently
 - Header files can be organized differently
 - All libraries may not be available everywhere
- ▶ **Standardized** build procedure
 - Standard options
 - Standard environment variables
 - Standard behavior



Alternatives to *autotools*

▶ Regular *Makefiles*

- Not very portable
- No configuration tests, or options
- Hard to take into account all dependencies (e.g. dependencies on header files)
- No standardized behavior

▶ CMake

- A more modern build system
- One language, instead of several for *autotools*
- More straightforward to use and understand
- Much less widely used than *autotools*, but growing in popularity
- Also generates Makefiles, like *autotools*

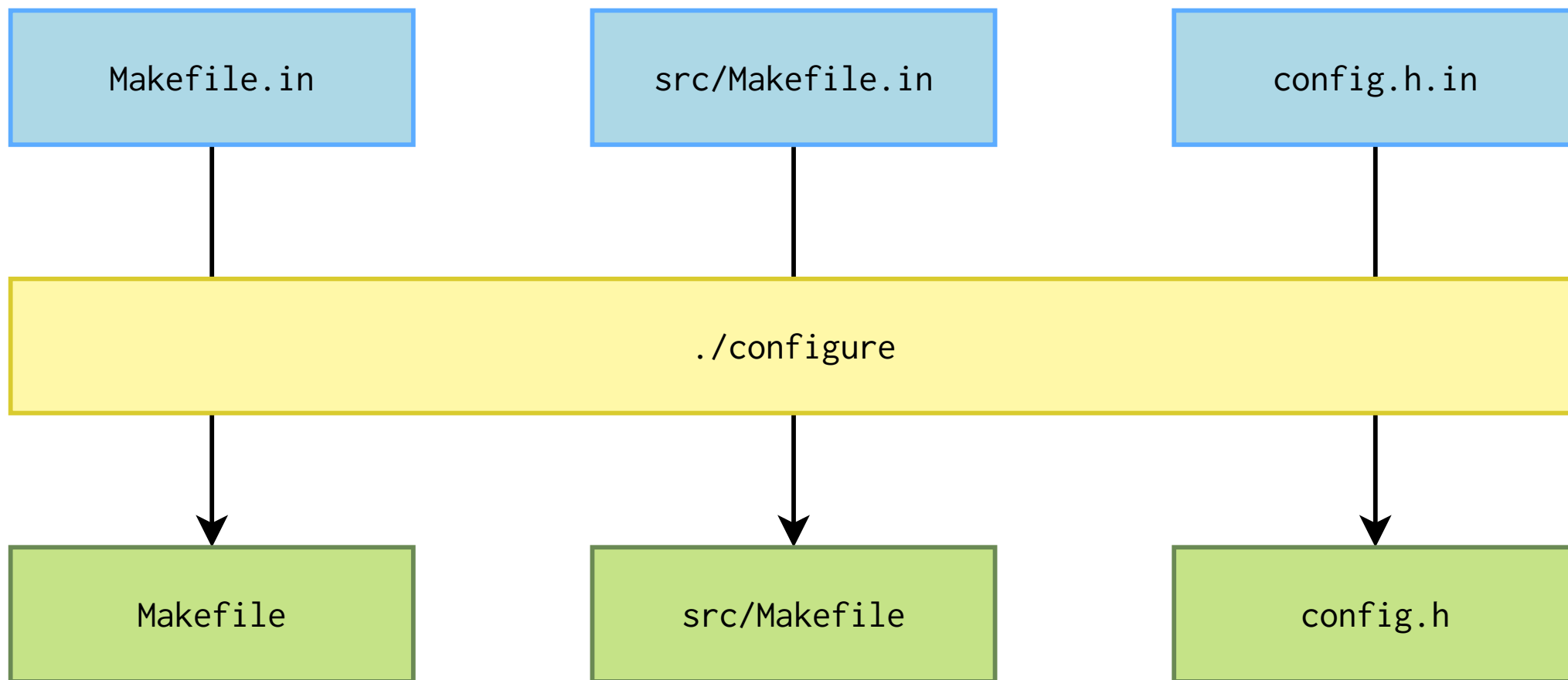


Using *autotools* based packages

- ▶ The basic steps to build an *autotools* based software component are:
 1. **Configuration** `./configure` Will look at the available build environment, verify required dependencies, generate `Makefiles` and a `config.h`
 2. **Compilation** `make` Actually builds the software component, using the generated `Makefiles`.
 3. **Installation** `make install` Installs what has been built.



What is `configure` doing?





Standard Makefile targets

- ▶ `all`, builds everything. The default target.
- ▶ `install`, installs everything that should be installed.
- ▶ `install-strip`, same as `install`, but then strips debugging symbols
- ▶ `uninstall`
- ▶ `clean`, remove what was built
- ▶ `distclean`, same as `clean`, but also removes the generated *autotools* files
- ▶ `check`, run the test suite
- ▶ `installcheck`, check the installation
- ▶ `dist`, create a tarball



Standard filesystem hierarchy

- ▶ `prefix`, defaults to `/usr/local`
 - `exec-prefix`, defaults to `prefix`
 - `bindir`, for programs, defaults to `exec-prefix/bin`
 - `libdir`, for libraries, defaults to `exec-prefix/lib`
- ▶ `includedir`, for headers, defaults to `prefix/include`
- ▶ `datarootdir`, defaults to `prefix/share`
 - `datadir`, defaults to `datarootdir`
 - `mandir`, for man pages, defaults to `datarootdir/man`
 - `infodir`, for info documents, defaults to `datarootdir/info`
- ▶ `sysconfdir`, for configuration files, defaults to `prefix/etc`
- ▶ `—<option>` available for each of them
 - E.g: `./configure --prefix= /sys/`



Standard configuration variables

- ▶ `CC`, C compiler command
- ▶ `CFLAGS`, C compiler flags
- ▶ `CXX`, C++ compiler command
- ▶ `CXXFLAGS`, C++ compiler flags
- ▶ `LDFLAGS`, linker flags
- ▶ `CPPFLAGS`, C/C++ preprocessor flags
- ▶ and many more, see `./configure --help`
- ▶ E.g: `./configure CC=arm-linux-gcc`



System types: build, host, target

- ▶ *autotools* identify three **system types**:
 - **build**, which is the system where the build takes place
 - **host**, which is the system where the execution of the compiled code will take place
 - **target**, which is the system for which the program will generate code. This is only used for compilers, assemblers, linkers, etc.
- ▶ Corresponding **-build**, **-host** and **-target** *configure* options.
 - They are all automatically *guessed* to the current machine by default
 - **-build**, generally does not need to be changed
 - **-host**, must be overridden to do cross-compilation
 - **-target**, needs to be overridden if needed (to generate a cross-compiler, for example)
- ▶ Arguments to these options are *configuration names*, also called *system tuples*



System type: *configuration names*

- ▶ A string identifying a combination of architecture, operating system, ABI and C library
- ▶ General format: `<arch>-<vendor>-<kernel>-<operating_system>`
 - `<arch>` is the type of processor, i.e. `arm`, `i686`, etc.
 - `<vendor>` is a free form string, which can be omitted
 - `<kernel>` is always `linux` when working with Linux systems, or `none` for bare metal systems
 - `<operating_system>` generally identifies the C library and ABI, i.e. `gnu`, `gnueabi`, `eabi`, `gnueabihf`, `uclibcgnueabi`
- ▶ Also often used as the *prefix* for cross-compilation tools.
- ▶ Examples
 - `x86_64-amd-linux-gnu`
 - `powerpc-mentor-linux-gnu`
 - `armeb-linux-gnueabi`
 - `i486-linux-musl`



System type: native compilation example

```
$ ./configure
[...]  
checking build system type... x86_64-unknown-linux-gnu checking host system  
type... x86_64-unknown-linux-gnu checking for gcc... gcc  
[...]  
checking how to run the C preprocessor... gcc -E  
[...]
```



Cross-compilation

- ▶ By default, *autotools* will guess the **host** machine as being the current machine
- ▶ To cross-compile, it must be overridden by passing the **-host** option with the appropriate *configuration name*
- ▶ By default, *autotools* will try to use the cross-compilation tools that use the *configuration name* as their prefix.
- ▶ If not, the variables **CC**, **CXX**, **LD**, **AR**, etc. can be used to point to the cross-compilation tools.



System type: cross compilation example

```
$ which arm-linux-gnueabi-gcc
/usr/bin/arm-linux-gnueabi-gcc
$ ./configure --host=arm-linux-gnueabi-gcc
[...]
checking build system type... x86_64-unknown-linux-gnu checking host system
type... arm-unknown-linux-gnueabi-gcc checking for arm-linux-gnueabi-gcc...
arm-linux-gnueabi-gcc
[...]
checking how to run the C preprocessor... arm-linux-gnueabi-gcc -E
[...]
```



Out of tree build

- ▶ *autotools* support out of tree compilation by default
- ▶ Consists in doing the build in a directory separate from the source directory
- ▶ Allows to:
 - Build different configurations without having to rebuild from scratch each time.
 - Not clutter the source directory with build related files
- ▶ To use out of tree compilation, simply run the configure script from another empty directory
 - This directory will become the build directory



Out of tree build: example

```
strace-4.9 $ ls configure configure.ac Makefile.am system.c NEWS
AUTHORS COPYING file.c ioprio.c config.h strace-4.9 $ mkdir ../
strace-build-x86 ../strace-build-arm strace-4.9 $ cd ../strace-build-x86
strace-build-x86 $ ../strace-4.9/configure
[...]
strace-build-x86 $ make
[...]
strace-build-x86 $ cd ../strace-build-arm strace-build-arm $ ../strace-4.9/
configure --host=arm-linux-gnueabihf
[...]
strace-build-arm $ make
[...]
```



Diverted installation with DESTDIR

- ▶ By default, `make install` installs to the directories given in `-prefix` and related options.
- ▶ In some situations, it is useful to *divert* the installation to another directory
 - Cross-compilation, where the build machine is not the machine where applications will be executed.
 - Packaging, where the installation needs to be done in a temporary directory.
- ▶ Achieved using the `DESTDIR` variable.

```
strace-4.9 $ make DESTDIR=/tmp/test install
[...]
strace-4.9 $ find /tmp/test/ -type f
/tmp/test/usr/local/share/man/man1/strace.1
/tmp/test/usr/local/bin/strace-log-merge
/tmp/test/usr/local/bin/strace-graph
/tmp/test/usr/local/bin/strace
```




–prefix or DESTDIR?

- ▶ –prefix and DESTDIR are often misunderstood
- ▶ –prefix is the location where the programs/libraries will be placed when executed on the *host machine*
- ▶ DESTDIR is a way of temporarily diverting the installation to a different location.
- ▶ For example, if you use –prefix=/home/<foo>/sys/usr, then binaries/libraries will look for icons in /home/<foo>/sys/usr/share/icons
 - Good for native installation in /home/<foo>/sys
 - **Bad** for cross-compilation where the binaries will ultimately be in /usr



–prefix or DESTDIR use cases

- ▶ Native compilation, install system-wide in /usr

```
$ ./configure --prefix=/usr  
$ make  
$ sudo make install
```

- ▶ Native compilation, install in a user-specific directory:

```
$ ./configure --prefix=/home/<foo>/sys/  
$ make  
$ make install
```

- ▶ Cross-compilation, install in /usr, diverted to a temporary directory where the system for the target is built

```
$ ./configure --prefix=/usr  
$ make  
$ make DESTDIR=/home/<foo>/target-rootfs/ install
```



Analyzing issues

- ▶ `autoconf` keeps a log of all the tests it runs in a file called `config.log`
- ▶ Very useful for analysis of `autoconf` issues
- ▶ It contains several sections: *Platform*, *Core tests*, *Running config.status*, *Cache variables*, *Output variables*, *confdefs.h*
- ▶ The end of the *Core tests* section is usually the most interesting part
 - This is where you would get more details about the reason of the *configure* script failure
- ▶ At the beginning of `config.log` you can also see the `./configure` line that was used, with all options and environment variables.



config.log example

```
$ ./configure ...
[...]
checking for TIFFFlushData in -ltiff34... no configure: WARNING: *** TIFF loader will not be built (TIFF library not found)
***
configure: error:
*** Checks for TIFF loader failed. You can build without it by passing
*** --without-libtiff to configure but some programs using GTK+ may
*** not work properly

$ cat config.log
[...]
configure:18177: .../usr/bin/x86_64-linux-gcc -std=gnu99 -o conftest -D_LARGEFILE_SOURCE
-D_LARGEFILE64_SOURCE -D_FILE_OFFSET_BITS=64 -O3 -static -Wall -D_LARGEFILE_SOURCE
-D_LARGEFILE64_SOURCE -D_FILE_OFFSET_BITS=64 -DG_DISABLE_SINGLE_INCLUDES -static
conftest.c -ltiff34 -ljpeg -lz -lm >&5
.../host/opt/ext-toolchain/bin/.../lib/gcc/x86_64-buildroot-linux-uclibc/4.8.4/../../../../
x86_64-buildroot-linux-uclibc/bin/ld: cannot find -ltiff34
.../host/opt/ext-toolchain/bin/.../lib/gcc/x86_64-buildroot-linux-uclibc/4.8.4/../../../../
x86_64-buildroot-linux-uclibc/bin/ld: cannot find -ljpeg collect2: error: ld returned 1 exit status configure:18177: $?
= 1
configure: failed program was:
[...]
configure:18186: result: no configure:18199: WARNING: *** TIFF loader will not be built (TIFF library not found) ***
configure:18210: error:
*** Checks for TIFF loader failed. You can build without it by passing
*** --without-libtiff to configure but some programs using GTK+ may
*** not work properly
```



autotools: *autoconf* and *automake*

- ▶ The `configure` script is a shell script generated from `configure.ac` by a program called `autoconf`
 - `configure.ac` used to be named `configure.in` but this name is now deprecated
 - written in shell script, augmented with numerous *m4* macros
- ▶ The `Makefile.in` are generated from `Makefile.am` files by a program called `automake`
 - Uses special `make` variables that are expanded in standard `make` constructs
- ▶ Some auxiliary tools like `autoheader` or `aclocal` are also used
 - `autoheader` is responsible for generating the *configuration header* template, `config.h.in`
- ▶ Generated files (`configure`, `Makefile.in`, `Makefile`) should not be modified.
 - Reading them is also very difficult. Read the real source instead!



Cache variables

- ▶ Each test done by a `configure.ac` script is associated with a *cache variable*
- ▶ The list of such variables and their values is visible in `config.log`:

```
## ----- ##  
## Cache variables. ##  
## ----- ##  
ac_cv_build=x86_64-unknown-linux-gnu ac_cv_c_compiler_gnu=yes  
[...]  
ac_cv_path_SED=/bin/sed
```

- ▶ If the autodetected value is not correct for some reason, you can override any of these variables in the environment:

```
$ ac_cv_path_SED=/path/to/sed ./configure
```

- ▶ This is sometimes useful when cross-compiling, since some tests are not always cross-compilation friendly.



- ▶ In general:
 - When a software is published as a *tarball*, the `configure` script and `Makefile.in` files are already generated and part of the tarball.
 - When a software is published through *version control system*, only the real sources `configure.ac` and `Makefile.am` are available.
- ▶ There are some exceptions (like tarballs not having pre-generated `configure/Makefile.in`)
- ▶ Do not version control generated files!



Regenerating *autotools* files: `autoreconf`

- ▶ To generate all the files used by *autotools*, you could call `automake`, `autoconf`, `aclocal`, `autoheader`, etc. manually.
 - But it is not very easy and efficient.
- ▶ A tool called `autoreconf` automates this process
 - Useful option: `-i` or `-install`, to ask `autoreconf` to copy missing auxiliary files
- ▶ Always use `autoreconf`!



autoreconf example

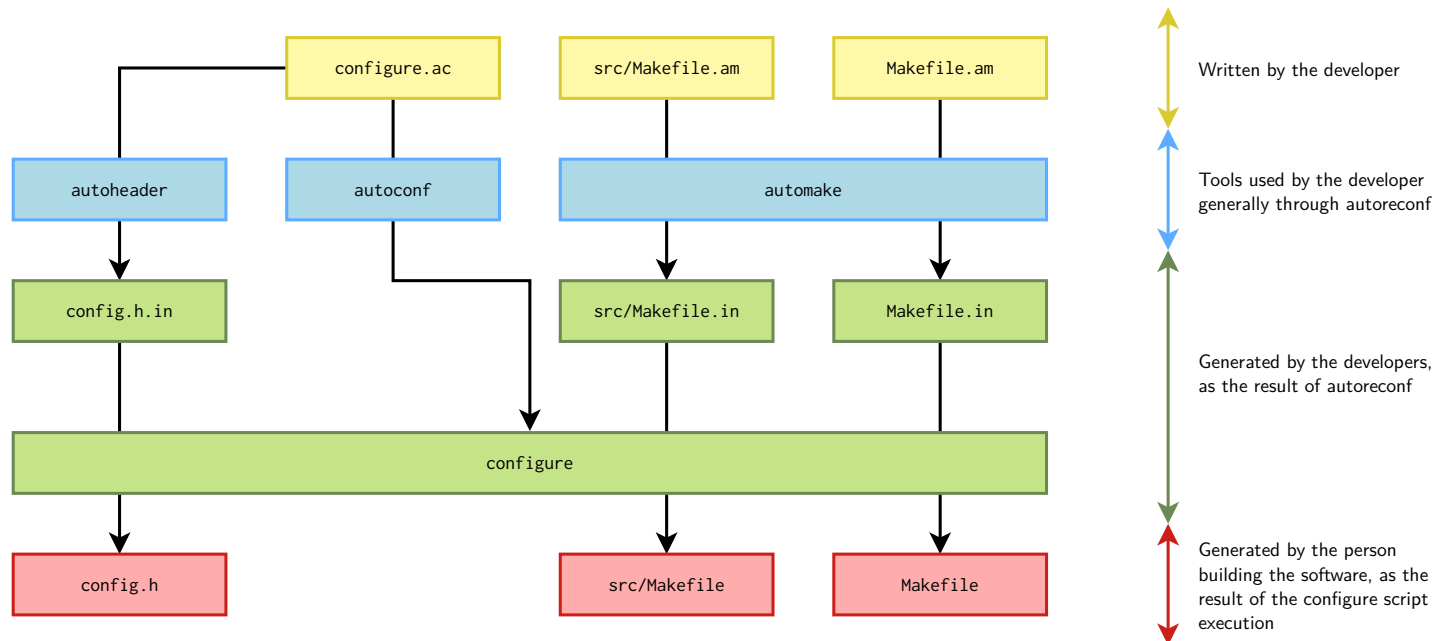
```
$ find . -type f
./src/main.c
./Makefile.am
./configure.ac

$ autoreconf -i configure.ac:4: installing './compile'
configure.ac:3: installing './install-sh'
configure.ac:3: installing './missing'
Makefile.am: installing './depcomp'

$ find . -type f
./install-sh
./src/main.c
./config.h.in
./configure
./missing
./depcomp
./aclocal.m4
./Makefile.am
./autom4te.cache/traces.0
./autom4te.cache/output.1
./autom4te.cache/output.0
./autom4te.cache/requests
./autom4te.cache/traces.1
./compile
./Makefile.in
./configure.ac
```



Overall organization



Usage of existing *autotools* projects

- ▶ First build of an *autotools* package
- ▶ Out of tree build and cross-compilation
- ▶ Overriding cache variables
- ▶ Using *autoreconf*